Journal of Computational Science 72 (2023) 102109

Contents lists available at ScienceDirect

Journal of Computational Science

journal homepage: www.elsevier.com/locate/jocs

SRL-assisted AFM: Generating planar unstructured quadrilateral meshes with supervised and reinforcement learning-assisted advancing front method

Hua Tong [a], Kuanren Qian [a], Eni Halilaj [a,b,c], Yongjie Jessica Zhang [a,b,*]

[a] Department of Mechanical Engineering, Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213, USA
[b] Department of Biomedical Engineering, Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213, USA
[c] Robotics Institute, Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213, USA

ARTICLE INFO

Dataset link: https://github.com/CMU-CBML/SRL-AssistedAFM, 10.5281/zenodo.8164390

Keywords:
Quadrilateral mesh generation
Complex geometry
Advancing front method
Supervised learning
Reinforcement learning

ABSTRACT

High-quality mesh generation is the foundation of accurate finite element analysis. Due to the vast interior vertices search space and complex initial boundaries, mesh generation for complicated domains requires substantial manual processing and has long been considered the most challenging and time-consuming bottleneck of the entire modeling and analysis process. In this paper, we present a novel computational framework named "SRL-assisted AFM" for meshing planar geometries by combining the advancing front method with neural networks that select reference vertices and update the front boundary using "policy networks". These deep neural networks are trained using a unique pipeline that combines supervised learning with reinforcement learning to iteratively improve mesh quality. First, we generate different initial boundaries by randomly sampling points in a square domain and connecting them sequentially. These boundaries are used for obtaining input meshes and extracting training datasets in the supervised learning module. We then iteratively improve the reinforcement learning model performance with reward functions designed for special requirements, such as improving the mesh quality and controlling the number and distribution of extraordinary points. Our proposed supervised learning neural networks achieve an accuracy higher than 98% on predicting commercial software. The final reinforcement learning neural networks automatically generate high-quality quadrilateral meshes for complex planar domains with sharp features and boundary layers.

1. Introduction

As outlined in NASA's Vision 2030, mesh generation constitutes one of the six crucial research directions and holds significance in numerical simulations [1]. However, mesh generation remains a major bottleneck due to algorithmic complexity, poor error estimation capabilities, and intricate geometries [2,3]. Quadrilateral (Quad) meshes are typically chosen over triangular meshes in applications such as texturing, simulation using finite elements, and B-spline fitting due to their attractive tensor-product nature and smooth surface approximation. Quad mesh generation has been a significant research topic for decades. Yet, existing quad mesh generation techniques rely heavily on pre-processing or post-processing to maintain good mesh quality and require heuristic expertise in algorithm construction. Pre-processing involves creating optimal vertex locations [4] and breaking down complex domains into regular components [5]. Post-processing is performed to clean inverted or irregularly connected elements. The operations include mesh adaptation [6], splitting, swapping, and collapsing elements as iterative topological alterations [7], as well as singularity reduction [8]. However, these additional mesh quality improvement procedures are computationally complex and inefficient.

There are two types of quad meshes: structured [9] and unstructured [10,11]. All elements in a structured mesh are arranged in a regular pattern before being mapped to the user-defined boundaries. The resulting mesh quality may be poor for complex boundaries. In unstructured meshes, interior node valence numbers are relaxed, allowing for greater flexibility in mesh construction. There are two methods for generating unstructured quad meshes: indirect and direct. The indirect method divides each triangle into three quads, which yields a lot of irregular nodes. An alternate algorithm is to combine adjacent pairs of triangles to form a single quad, which improves mesh quality but leaves some triangles [10]. An advancing front method (AFM) is proposed to improve the latter and generate an all-quad mesh from triangles [12]. The initial front of the mesh is defined by delineating the triangle edges at the boundary. A sequence of paired triangles is systematically merged along the front, progressively moving toward the interior. Q-Morph is another AFM-based method that can effectively decrease the

* Corresponding author at: Department of Mechanical Engineering, Carnegie Mellon University, 5000 Forbes Ave, Pittsburgh, PA 15213, USA.
E-mail address: jessicaz@andrew.cmu.edu (Y.J. Zhang).

https://doi.org/10.1016/j.jocs.2023.102109
Received 30 April 2023; Received in revised form 8 July 2023; Accepted 11 July 2023
Available online 23 July 2023

number of irregular nodes by performing local edge swapping and inserting additional nodes [13]. However, indirect methods require an intermediate triangular mesh, which is prone to instability and has a restricted number of vertices. The direct method, on the other hand, bypasses triangulation entirely and instead generates quad elements directly, avoiding those potential problems.

Many direct methods have been proposed in recent years. The paving method initiates from the boundary and proceeds inward by arranging complete rows of elements on the front boundary once at a time [14]. A subdivision-based quadrangulation algorithm is presented for exhaustively enumerating topologies within each patch after segmenting the input boundary into several surface patches [15]. The quadtree/hexagon-based methods are hierarchical approaches to mesh generation subdividing a region into quad [16,17] or hexagonal [18] cells recursively based on geometric criteria. In [19], an individual interior or exterior mesh is generated at a time and matched at the shared boundary. Mesh quality can be improved via face swapping, edge removal, and geometric flow-based smoothing [20]. Octree-based iso-contouring methods analyze each interior grid vertex and generate a dual mesh of the background grids for any complicated single-material and multiple-material domains [21–23]. Several parameterization-based quad remeshing methods have also been proposed. Discrete harmonic scalar function methods compute piecewise smooth harmonic scalar functions to tile the input surface into quadrangles using isolines with [24] or without [25] any T-junctions. A cross field method is proposed to capture the geometric structure of the input surface [26]. Optimizations in complexity and singularity separating conditions are proposed to tackle degeneracies on real-world input data and enable global search for high-quality coarse quad layouts [27]. In biomedical applications, vascular blood flow simulation using isogeometric analysis (IGA) needs high-order elements like T-splines [28], subdivisions [29,30], and THB-splines [31]. All prefer high-quality unstructured quad and hexahedral meshes because they can be converted into standard or rational T-splines, which possess C^2-continuity across the entirety of the surface, excluding local regions proximal to the extraordinary points (EPs, defined as either interior vertices with a valence $\neq 4$ that are not T-junctions or boundary vertices with a valence >3) [32,33]. Together with local refinement in IGA, the computational efficiency can be greatly improved.

Our research is based on the AFM, one of the most widely-used direct methods [7]. AFM is a greedy algorithm that iteratively creates mesh nodes from the input boundaries to the interior. Each iterative process consists of three steps: (1) selecting a line segment from the front set that separates the meshed domain and the unmeshed domain; (2) connecting a new mesh node or existing mesh nodes to the base segment to generate a high-quality quad element; and (3) updating the front set until the entire domain is meshed. AFM can produce a high-quality mesh, but it is inefficient since it requires numerous intersection calculations [34,35]. Several literatures attempt to integrate mesh generation with machine learning (ML) modules to create new meshing algorithms. Training a reinforcement learning neural network on an input boundary multiple episodes to generate a final good-quality mesh [36] is one example. Deep learning is also used to learn the progress direction and step size of triangle mesh generation [37]. However, these methods require intersection detection in the middle stage or after mesh generation, which is computationally expensive. In addition, these methods were only tested on simple boundaries and did not provide open-source codes to test their generalization for complex domains.

To tackle complex boundaries and satisfy special requirements in generating quad meshes, we propose a new computational framework named "SRL-assisted AFM" for quad mesh generation using AFM assisted by supervised learning (SL) and reinforcement learning (RL). We train the SL module with the dataset extracted from input meshes. The RL module automatically generates high-quality meshes with designed reward functions for various complex geometries and back-propagates neural network weights based on high-quality training datasets. The proposed SRL-assisted AFM framework is capable of meshing complex new boundaries efficiently without any topological restrictions on the input domain. Users can also add boundary layers and optimize the number and distribution of EPs in the mesh. The main contributions of this paper include:

- Integrating AFM, SL, and RL into a new computational framework to generate high-quality quad meshes for planar domains with complex boundaries;
- Eliminating the need for quality improvement and intersection detection during and after mesh generation; and
- Preserving high Jacobian, low aspect ratio, low EP number with sharp features, unbalanced seeds, conformal boundaries, and boundary layers.

The remainder of this paper is structured as follows. In Section 2, we overview the comprehensive framework of AFM mesh generation as an SL-RL problem. In Section 3, we discuss the detailed AFM, SL, and RL modules, along with the comprising action, state, and reward settings. In Section 4, we present numerical results and discuss our findings. We conclude by summarizing our contributions and proposing future work in Section 5.

2. Overview of the SRL-assisted AFM framework

We combine the AFM with the SL and RL modules to automatically generate high-quality quad meshes (Fig. 1). AFM generates one quad element at a time based on the front boundary information and evolves the front at each time step. The meshing process is complete when the evolving front becomes a quad element. In the literature, rule-based algorithms are adopted at each step to generate a new element [34,38]. Here we replace rule-based algorithms with policy neural networks (SL and RL), which is capable of approximating any complex functions [39].

Our SRL-assisted AFM framework (Fig. 1(a)) includes a training procedure (blue arrows) and a testing procedure (black arrows). In the training procedure, we first randomly generate 360 planar training boundaries by connecting randomly placed vertices in a square domain. Then, we use commercial software ANSYS to generate quad meshes based on these boundaries and collect $3.5M$ quad elements for our training dataset. The four SL policy neural networks (Fig. 1(b)) $\{\pi_a, \pi_b, \pi_c, \pi_d\}$ take local information around a selected vertex as input and work together sequentially to update the front: (1) collecting local information from the vertex with the smallest angle on the front, (2) sending local information to π_a and selecting the reference point, (3) sending reference point local information to π_b and getting the updating type, and (4) sending reference point local information to π_c and π_d to generate new interior vertices if the π_b updating type requires inserting new points. We use input meshes results to obtain the optimized policy network weights via AFM and SL to circumvent random weights-induced poor performance.

To further improve the framework, we combine AFM with RL neural networks (Fig. 1(c)). We consider the AFM as a Partially Observable Markov Decision Process [40] and combine it with RL. We transfer trained SL neural networks to RL neural networks with the same architecture denoted as $\{\pi'_a, \pi'_b, \pi'_c, \pi'_d\}$ and train them on 5 randomly selected initial boundaries (Fig. 1(a)). At each time step t during the RL training process, the environment (current front) sends a state S_t to the neural network. Then the neural network samples an updating action $A(S_t)$ and receives a reward (mesh quality metrics, Fig. 1(c)) as feedback of its action. In our implementation, the reward function measures the squareness of elements as well as the penalty of EPs: The higher the squareness reward, the closer a quad element is to a square. For the EP penalty, all regular vertices receive reward "1", while EPs receive penalty "0". A pair of EPs that are adjacent to each other are denoted as close EPs or cEPs. All cEPs also receive the "0"

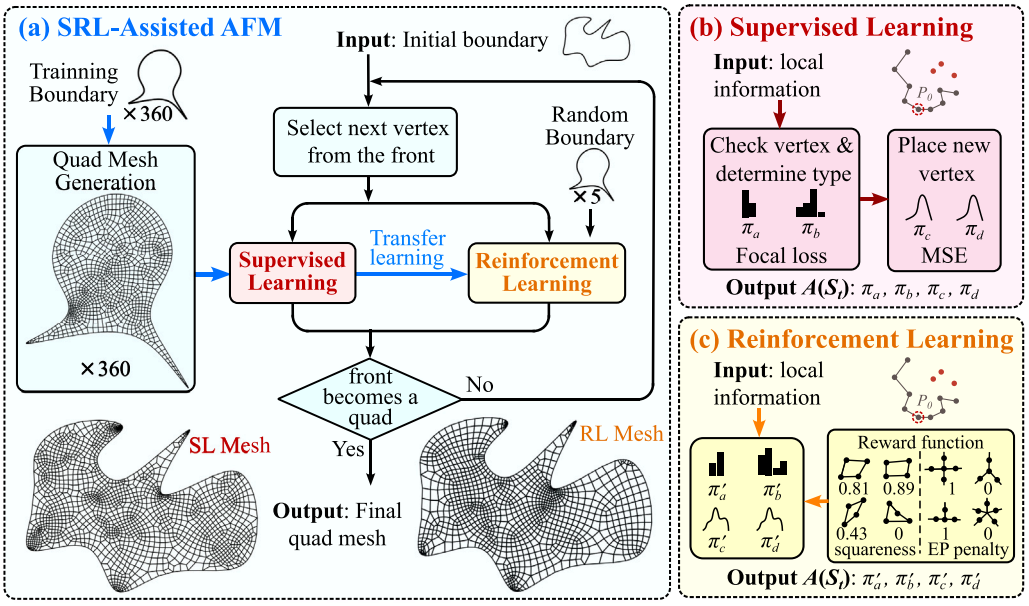


Fig. 1. Neural network pipeline and architecture. (a) The SRL-assisted AFM framework utilizes SL and RL neural networks to generate new vertices following the AFM scheme. (b) SL neural networks learn from ANSYS-generated quad meshes and generate a new quad on the front. (c) RL neural networks use squareness and EP penalty as the reward functions and are trained on the evolving front to maximize the reward feedback.

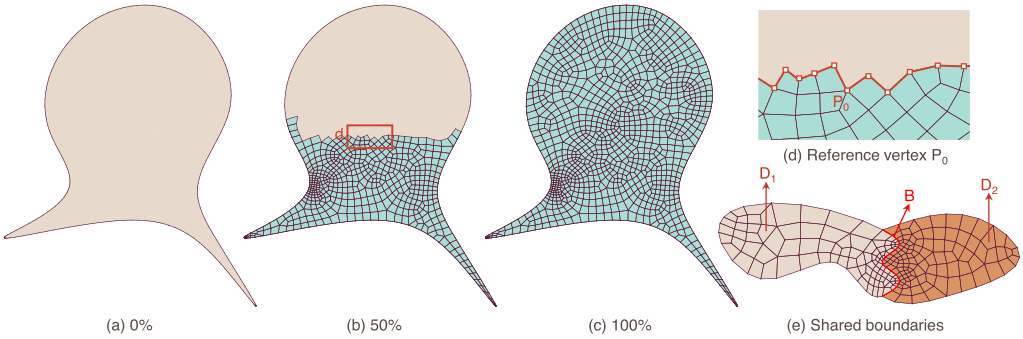


Fig. 2. Meshing a planar domain with advancing front method. (a) The initial domain boundary. (b) The meshing process at 50% completeness. (c) The final mesh. (d) A zoom-in picture of the reference vertex (P_0) and the front (red line) at 50% completeness. (e) Two domains (D_1 in beige and D_2 in orange) with a shared boundary (B in red).

penalty. These three steps (receiving the local information, sampling actions with neural network predictions, and updating the front) are iterated until the evolving front becomes a quad, signaling the meshing process is completed. As we mesh the same domain multiple times, the mesh (dataset) quality improves, and we update the neural network weights with the latest high-quality dataset. In the end, we obtain RL neural networks and meshes superior over input meshes in various mesh quality metrics.

3. Methodology

Our SRL-assisted AFM pipeline combines three fundamental modules (AFM, SL and RL) together. The technical details of each module are explained below.

3.1. SRL-assisted AFM

In the paper we employ AFM, which iteratively generates one quad element at a time to fill the entire domain (Fig. 2). Our implementation is shown in Algorithm 1. It takes planar closed manifold boundaries $B_i^0, i = 1, 2, \ldots, L$ as input and outputs an all-quad mesh. These L closed manifold boundaries may contain sharp features. After specifying the initial boundary, we assign seeds to the boundary based on a seed interval function $s(i)$, which is proportional to the local boundary curvature $\rho(i)$. We use the circumcircle of three local vertices P_{i-1}, P_i, and P_{i+1} to estimate $\rho(i)$. Then we obtain $s(i) = k(i)\rho(i)$, and $k(i)$ is calculated by

$$k(i) = \frac{l_{tot}}{s_a \sum_{i=1}^{N} \frac{l(i)}{\rho(i)}}, \quad where\ \rho(i) \neq 0. \tag{1}$$

s_a is the average sampling seed interval (the default $s_a = 0.05$). We calculate the whole front boundary length $l_{tot} = \sum_{i=1}^{N} l_i$, where l_i is the length of edge $P_i P_{i+1}$.

The seed assignment procedure supports multiple domains with shared boundaries. Suppose we have two domains D_1 and D_2 with boundaries B_1 and B_2, respectively, as in Fig. 2(e). The shared boundary B is part of B_1 and B_2, or $B_1 \cap B_2 = B$. At a certain node on B numbered i on B_1 and numbered j on B_2, the local front boundary curvature on both boundaries is the same, $\rho_1(i) = \rho_2(j)$, because the three local vertices are on the shared boundary B. However, the coefficients on B_1 and B_2 are different or $k_1(i) \neq k_2(j)$ because B_1 and B_2 may have different l_{tot} and s_a values. We assign the new coefficient $\frac{k_1(i)+k_2(j)}{2}$ to this node on the shared boundary B. In this way, seeds on both boundaries conform exactly to each other.

In the final step, we check if each of these L initial boundaries has an even number of edges. If not, we insert a new seed point to

Algorithm 1 SRL-assisted AFM

Input: Planar closed manifold boundaries $B_i^0, i = 1, 2, \cdots, L$
Output: Interior or exterior quad meshes
Goal: Fill the interior or exterior domain enclosed by $B_i^0, i = 1, 2, \cdots, L$ with all quad elements

for boundary number $i = 1, 2, \cdots, L$ **do**
  Calculate local boundary curvature $\rho(i)$ and coefficient $k(i)$
  Assign adaptive seeds to B_i^0 according to the seed interval function$s(i) = k(i)\rho(i)$
  for boundary number $j = 1, 2, \cdots, L, j \neq i$ **do**
    if B_i^0 and B_j^0 have a shared boundary B **then**
      Calculate adaptive mesh coefficients on B (denoted as k_i and k_j)
      Assign new seeds with coefficients $k_{new} = \frac{k_i + k_j}{2}$ to B
    end if
  end for
end for
for time step $t = 1, 2, \cdots, M$ **do**
  for boundary number $i = 1, 2, \cdots, L$ **do**
    Define N as the number of vertices on the remaining boundary B_i^t
    if $N > 4$ **then**
      for vertex number $j = 1, 2, \cdots, N$ **do**
        Save local information around vertex P_j from B_i^t as input tensor S_j
        Input S_j to π_a (or π_a') to judge whether P_j can be the reference vertex
      end for
      Denote the selected reference vertex and its local information as P_0, S_t
      Input S_t to π_b (or π_b') to obtain the updating type T_t
      if $T_t = 1$ **then**
        Input S_t to π_c (or π_c') to obtain one new vertex P_1^{new}
      end if
      if $T_t = 4$ **then**
        Input S_t to π_d (or π_d') to obtain two new vertices $P_{1,2}^{new}$
      end if
      Form a new quad element and update front boundary B_i^t
    end if
  end for
end for
Add boundary layers to specified initial manifold boundaries

the identified initial boundary to guarantee that the final meshes are all-quad. After assigning the seeds, we begin to mesh the domain. In Algorithm 1, a constant $M = 100{,}000$ represents the maximum allowed time step of AFM. At each time step t, we select a reference vertex P_0 from the front boundary (the reference vertex position determines where we update the front boundary), collect local information S_t around P_0, and take action $A(S_t)$ to update the front. In our SRL-assisted AFM framework, we have four neural networks $\{\pi_a, \pi_b, \pi_c, \pi_d\}$ in SL and $\{\pi_a', \pi_b', \pi_c', \pi_d'\}$ in RL that identify the proper P_0 and generate a new quad around P_0 at each time step. After iterating through all vertices on the front, a vertex is chosen as the reference P_0 based on the judgment given by the binary classification neural network (π_a in SL, π_a' in RL). As shown in Fig. 3(a), π_a accepts P_0 to be the reference vertex for these four cases, which corresponds to the four updating types. Note that Types 1 and 4 insert one and two new vertices, respectively, and Types 2 and 3 connect two existing vertices on the front. Fig. 3(b) show six example cases, where π_a rejects the point when the formed quad element separates the domain into two subdomains or only one new point is generated and connects with P_0 (introducing more complex cases). After this, an updating type is selected based on the four-class classification neural network (π_b in SL, π_b' in RL). π_c and π_d (π_c' and π_d' in RL) are then used to insert one or two vertices in the domain to form a new quad element based on selected types. The meshing process is done only when the number of edges on the front becomes 4, forming the last quad.

During the mesh generation process, when the updated line segments intersect with the current front, we can correct the error by partitioning the front into two new fronts. Fig. 4 shows a local region of the evolving front, and neural network π_b (or π_b') selects Type 2 or Type 3 classification. Normally, we connect line segment $P_{i+1}P_{i-2}$ (the blue line segment) that corresponds to Type 2, or $P_{i+2}P_{i-1}$ that corresponds to Type 3. However, if $P_{i+1}P_{i-2}$ intersects with the remaining front boundary ($P_{j-1}P_j$ and $P_{j+1}P_j$), we partition the front boundary into two new front boundaries which form two subdomains. Eventually, Algorithm 1 will fill these two subdomains with quad elements. Theorem 1 discusses special intersection situations when separating the original front boundary into two boundaries is needed to continue the meshing process.

Theorem 1. *$\forall$ Planar domain D_0 with an even number of edges $N_0 \geq 4$, $\exists$ a partition method to split D_0 into two subdomains D_1, D_2, both of which have an even number of edges N_1 and N_2, satisfying $N_1 + N_2 - 2 = N_0$.*

Proof. In Fig. 4, assume there are multiple line segments ($P_{j-1}P_j$ and P_jP_{j+1}) on D_0 that intersect with $P_{i-2}P_{i+1}$, we can define a set $\mathcal{P}$ such that all $P_j \in \mathcal{P}$. In set $\mathcal{P}$, we can always find a vertex P_j that is the closest to the line segment P_iP_{i-1}:

$$P_j = \underset{j}{\operatorname{argmin}}\, dist(P_j, P_iP_{i-1}) = \underset{j}{\operatorname{argmin}} \frac{\left|\overrightarrow{P_iP_{i-1}} \times \overrightarrow{P_iP_j}\right|}{|\overrightarrow{P_iP_{i-1}}|}. \tag{2}$$

There exist no further line segments on D_0 that intersect with P_iP_j or $P_{i-1}P_j$, which means D_0 can be partitioned along either P_iP_j or $P_{i-1}P_j$. We partition D_0 into two subdomains along P_iP_j or $P_{i-1}P_j$. The operation forms two subdomains with the number of edges N_1 and N_2, respectively. The choice of partitioning along P_iP_j or $P_{i-1}P_j$ depends on which choice ensures both N_1 and N_2 are even. Otherwise, D_1 and D_2 cannot be filled by all quads. After the partition, one new edge is created and shared by D_1 and D_2. Therefore, we have $N_1 + N_2 - 2 = N_0$. □

After the meshing, we can add boundary layers to the internal boundary for fluid mechanics simulations. We construct the boundary layer by splitting the elements along the boundary. Four templates are implemented (Fig. 5) for scenarios with one edge, one vertex, and two adjacent edges of an element on the boundary. As a result, they yield two, three, three, and four smaller elements, respectively. Note that Templates (b) and (c) introduce a new valence-3 EP. In Template (c), if we insert two new vertices on the boundary [17], we obtain Template (d), which could avoid introducing the new valence-3 EP.

3.2. Supervised learning

AFM's nature of using local information not only facilitates the implementation of neural networks for automating and enhancing the rule learning process but also makes it possible to extract datasets from a given mesh. We generate 360 input boundaries by sampling N points in a unit square domain, where $N \in [4, 100]$ and is an even number, and connect these points sequentially. To avoid invalid geometries, we conduct an intersection check for each newly connected line segment with existing edges. Once we obtain the valid domain boundaries, we use ANSYS to generate corresponding quad meshes for SL training.

The data for SL are input–output pairs extracted at each iteration from the meshes we obtained. In Fig. 1(b, c), the input tensor of the policy neural network contains 12 vertices: the reference vertex P_0, four vertices on the left $P_i^l, i = 1, 2, 3, 4$, four vertices on the right $P_i^r, i = 1, 2, 3, 4$, and the closest three red vertices $P_i^c, i = 1, 2, 3$, to P_0.

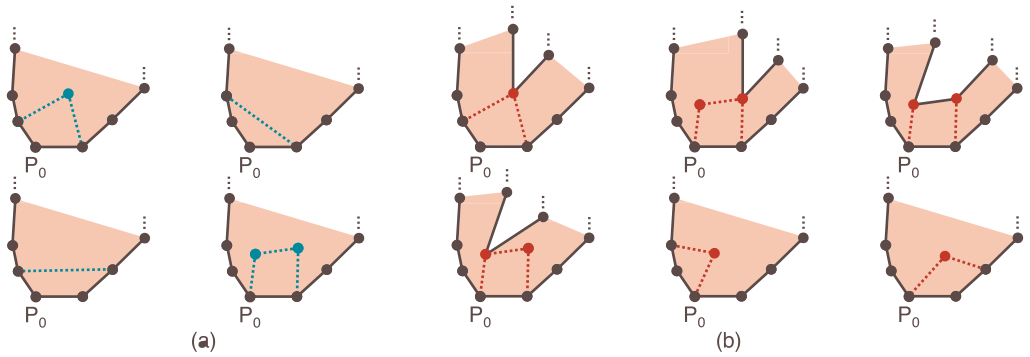


Fig. 3. The judgment principle of the binary classification neural network π_a when labeling the ground truth dataset. (a) Four cases (corresponds to the four updating types) when π_a accepts P_0 to be the reference vertex. (b) Six example cases when π_a rejects P_0 to be the reference vertex.

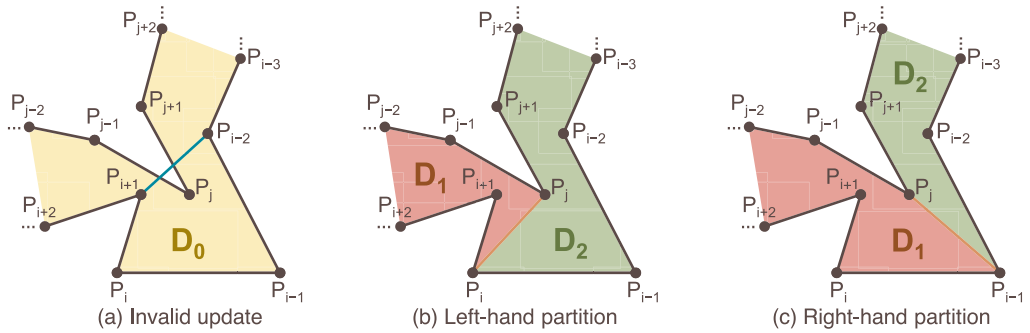


Fig. 4. Two special situations when a partition operation is necessary to split the domain D_0. (a) Assuming that P_i is the reference vertex, the next update, which seals the line segment $P_{i+1}P_{i-2}$, is invalid because $P_{i+1}P_{i-2}$ intersects with the remaining front boundary. (b) D_0 is partitioned along P_iP_j if the resulting two new subdomains D_1, D_2 both have an even number of edges. (c) D_0 is partitioned along $P_{i-1}P_j$ if the edge numbers of both new subdomains D_1, D_2 are even.

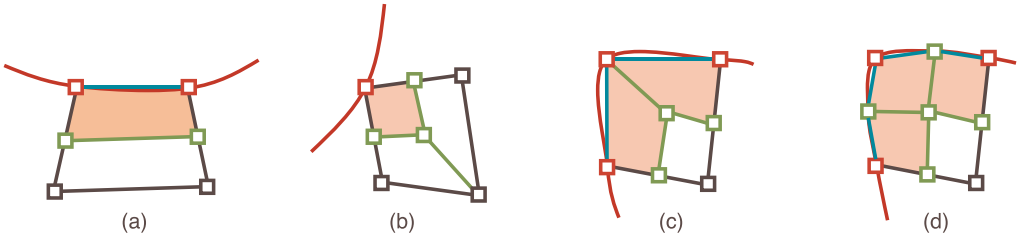


Fig. 5. Four templates of adding the boundary layer. The quad element shares one edge (a), one vertex (b) and two adjacent edges (c, d) with the boundary. A valence-3 EP is introduced in (b, c), while two new vertices are inserted on the boundary in (d).

Note that P_0, P_i^l, P_i^r and P_i^c are all on the front, and the closest three red vertices $P_i^c, i = 1,2,3$ are used for intersection checking. We also need valence information of vertices $P_0, P_i^r, P_i^l, i = 1,2$, because their valences may change during the iteration and turn them from regular vertices into EPs or even cEPs. We define the valence information of these five vertices as "EP status" to help neural networks distinguish EPs and cEPs from regular vertices. According to the definition of EPs in [28], we assign the EP status of an interior valence-V vertex to be $V-4$. If the vertex is on the input boundary, its EP status is -0.5 when $V < 2$ and $V-2$ otherwise. Then we have EP status of -1 (interior valence-3 EPs), -0.5 (regular boundary vertices), 0 (regular interior or boundary vertices), and >0 (EPs). Using these information, the SL and RL neural networks will try to avoid creating EPs when updating the front. There are four updating types for the front, as defined in Fig. 3(a). Type 1 generates a new vertex, and the number of edges in the front remains the same; Types 2 and 3 do not generate new vertices, and they remove two edges from the front; Type 4 generates two new vertices and inserts two new edges to the front. In the implementation, we reduce the input dimension by normalizing the coordinates. All vertices are transformed by normalizing P_0 and P_1^r to $(0,0)$ and $(1,0)$ using matrix transformation:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{1}{d} & 0 & 0 \\ 0 & \frac{1}{d} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}, \tag{3}$$

where $(x, y), (x', y')$ are the coordinates before and after transformation, θ is the angle between $P_0P_1^r$ and the horizontal x-axis, and d is the edge length $P_0P_1^r$.

In Fig. 6, we use four residual neural networks to mesh the domain for two reasons: (1) residual connections support very deep neural networks while avoiding the gradient vanishing and over-fitting problem; and (2) a residual block can be easily added to existing neural networks by initializing itself to be an identity mapping, which allows users to add more layers to the neural networks without requiring a complete overhaul of the architecture. The first neural network π_a determines whether to select a vertex as the reference P_0. All vertices on the front are arranged in ascending order according to the inner angle. We start from small to large and pass local information S_t around the selected vertex into the binary classification neural network π_a. The softmax layer in π_a gives a binary classification result $\{\mathscr{P}_{acc}, \mathscr{P}_{rej}\}$. The process

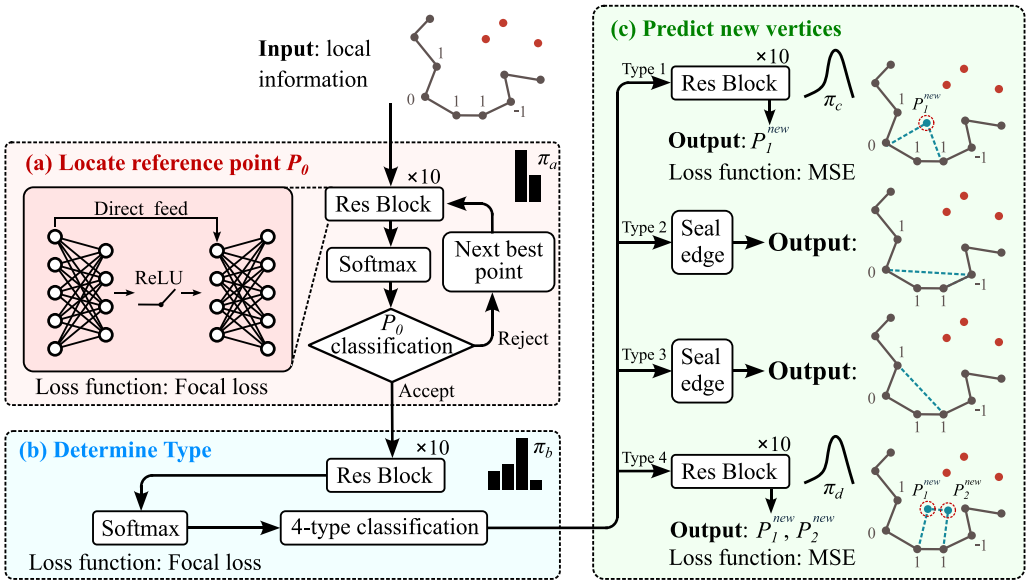


Fig. 6. The residual neural network framework [41] used in SL. (a) π_a is a binary classification neural network that determines the reference vertex P_0. (b) π_b is a four-class classification neural network that determines the updating type. (c) We then selectively apply π_c, π_d or seal the edge based on π_b classification results.

stops when π_a accepts the current vertex to be the reference vertex P_0. After we select P_0, the local information around P_0, S_t, is sent to π_b to determine the updating type. π_b is a four-class classifications neural network. The softmax layer in π_b gives a four-class classification result $\{\mathscr{P}_1, \mathscr{P}_2, \mathscr{P}_3, \mathscr{P}_4\}$ that selects one out of four updating types. Therefore, both π_a and π_b adopt Focal Loss [42] to address the class imbalance problem. For π_a and π_b, we have

$$FocalLoss_{\pi_{a/b}} = -\frac{1}{N_{a/b}}\alpha_{a/b}\sum_{i=1}^{N_{a/b}}\left[\left(1-p^i\right)^{\gamma}\log p^i\right], \tag{4}$$

where $N_{a/b}$ is the total number of neural network π_a or π_b training rows. For π_a, α_a is a scaling factor of two classes, p^i is the acceptance probability $\mathscr{P}^i_{acc}$ when π_a selects the correct category in the ground truth, and the rejection probability $\mathscr{P}^i_{rej}$ otherwise. For π_b, α_b is a scaling factor of four classes, p^i is the acceptance probability $\{\mathscr{P}^i_1, \mathscr{P}^i_2, \mathscr{P}^i_3, \mathscr{P}^i_4\}$ when the ground truth updating type is $1, 2, 3$ and 4, respectively. We have $\mathscr{P}^i_{acc} + \mathscr{P}^i_{rej} = 1$ and $\mathscr{P}^i_1 + \mathscr{P}^i_2 + \mathscr{P}^i_3 + \mathscr{P}^i_4 = 1$. γ is a tunable parameter that controls the weight of difficult-to-classify samples.

As shown in Fig. 6(c), when π_b gives Type 1 classification result, we send local information S_t to the regression neural network π_c to generate a new interior vertex P_1^{new}. We simply seal the edge when the updating type is 2 or 3, and no neural network is needed. When π_b gives Type 4 classification result, we send the same local information S_t to the regression neural network π_d to generate two new interior vertices P_1^{new} and P_2^{new}. For all P_1^{new} in π_c and P_1^{new}, P_2^{new} in π_d, we utilize their polar coordinates in the following loss function computation. With P_0 as the origin, the angle and radius are normalized to $[0, 1]$ by the reference vertex angle ($\angle P_1^r P_0 P_1^l$) and the total length of six line segments around P_0 with three on the left and three on the right ($\overline{P_3^r P_2^r} + \overline{P_2^r P_1^r} + \overline{P_1^r P_0} + \overline{P_0 P_1^l} + \overline{P_1^l P_2^l} + \overline{P_2^l P_3^l}$). Both π_c and π_d adopt MSE loss. For π_c we have

$$MSE_{\pi_c} = \frac{1}{N_c}\sum_{i=1}^{N_c}\left[\left(\theta_1^i - \hat{\theta}_1^i\right)^2 + \left(\rho_1^i - \hat{\rho}_1^i\right)^2\right], \tag{5}$$

where N_c is the total number of neural network π_c training rows. θ_1^i and $\hat{\theta}_1^i$ are the prediction and ground truth angle of P_1^{new}. ρ_1^i and $\hat{\rho}_1^i$ are the prediction and ground truth radius of P_1^{new}. Similarly for π_d we have

$$MSE_{\pi_d} = \frac{1}{N_d}\sum_{i=1}^{N_d}\left[\left(\theta_1^i - \hat{\theta}_1^i\right)^2 + \left(\rho_1^i - \hat{\rho}_1^i\right)^2 + \left(\theta_2^i - \hat{\theta}_2^i\right)^2 + \left(\rho_2^i - \hat{\rho}_2^i\right)^2\right], \tag{6}$$

where N_d is the total number of neural network π_d training rows. $\theta_1^i, \hat{\theta}_1^i$ and $\theta_2^i, \hat{\theta}_2^i$ are the prediction and ground truth angle of P_1^{new} and P_2^{new}. $\rho_1^i, \hat{\rho}_1^i$ and $\rho_2^i, \hat{\rho}_2^i$ are the prediction and ground truth radii of P_1^{new} and P_2^{new}. For all the neural networks, we use 10 residual blocks [41] as the middle layers. The size of the training data determines the performance of the model. In principle, the training data should cover the feasible region of the input features comprehensively. We feed 360 input domains or 3.5M training rows to the SL neural networks in the paper.

3.3. Reinforcement learning

To further improve the quad mesh quality generated by the framework, we fine-tune the framework by applying RL to the four trained SL neural networks $\{\pi_a, \pi_b, \pi_c, \pi_d\}$ using five randomly selected new boundaries. The fine-tuned RL neural networks $\{\pi'_a, \pi'_b, \pi'_c, \pi'_d\}$ have the same architecture as Fig. 6. The whole RL algorithm is demonstrated in Algorithm 2.

To obtain meshes with higher quality than the SL-generated meshes, we introduce additional exploration to the neural network-guided action by adding noise ϵ to RL neural networks. We add Dirichlet noise $Dir(\alpha)$, a type of probability distribution that assigns probabilities to an arbitrary number of outcomes, to classification neural networks π'_a and π'_b and $2-D$ Gaussian noise $\mathcal{N}(\mu, \Sigma)$ to regression neural networks π'_c and π'_d. As a supplement to the fifth line of Algorithm 2, we achieve this by sampling four actions from four different distributions. We have

$$A_t^{a/b} \sim \eta\pi'_{a/b}(S_t) + (1-\eta)Dir(\alpha^{a/b}), \tag{7}$$

$$A_t^{c/d} \sim \mathcal{N}\left(\pi'_{c/d}(S_t), \Sigma^{c/d}\right). \tag{8}$$

Based on several trials, we set $\eta = 0.99, \alpha^{a/b} = 0.05$, and $\Sigma^{c/d} = 0.0001 I_{2\times 2}$ to balance the neural network exploration and exploitation trade-off. Here $I_{2\times 2}$ is a 2×2 identity matrix.

The RL algorithm takes SL policy neural networks and five randomly selected initial boundaries as input and outputs fine-tuned RL policy neural networks. The neural network π'_a, at time step t, selects a vertex as the reference P_0 and records the state S_t describing vertices near P_0 from the environment. Then π'_b, π'_c or π'_d conduct an action $A(S_t)$ that determines the updating type and assigns new vertices (if any) to the environment. The environment responds to the action and transits into

Algorithm 2 Reinforcement Learning

Input: SL neural networks $\{\pi_a, \pi_b, \pi_c, \pi_d\}$, and 5 randomly selected boundaries B_i^0
Output: RL neural networks $\{\pi'_a, \pi'_b, \pi'_c, \pi'_d\}$
Goal: Improve the neural network meshing performance

Copy the SL neural networks and denote them as initial RL neural networks $\{\pi'_a, \pi'_b, \pi'_c, \pi'_d\}$
for episode $i = 1, 2, \cdots, M$ **do**
  Get an initial state S_1^{ij} (i is omitted hereafter)
  for time step $t = 1, 2, \cdots, M$ **do**
    $A_t \leftarrow$ sampling action $(\pi'_a, \pi'_b, \pi'_c, \pi'_d, S_t, \epsilon)$
    Form a new quad element and update the front boundary with A_t
    Get the reward (element quality) $R_t = R_t^s R_t^{ep}$ and the next state S_{t+1}
    if the number of edges on the remaining front boundary = 4 **then**
      Break
    end if
  end for
  if $R^{fin} = \frac{1}{M}\sum_{i=1}^{M} R_i + \min\{R_1, R_2, \cdots, R_M\} > R'_{fin}$ **then**
    Extract dataset from the new RL mesh
    Update $\{\pi'_a, \pi'_b, \pi'_c, \pi'_d\}$ weights with the new dataset
  end if
end for

a new state S_{t+1} at time step $t+1$. After meshing, we do not apply any post-processing. Finally, we calculate the reward related to the mesh quality. If the reward exceeds the previous mesh, we save the current mesh and extract the dataset.

The reward function is critical in guiding the mesh optimization direction in RL. In our framework, we have two reward functions: the squareness reward function R^s and the EP penalty reward function R^{ep}. R^s is defined based on inner angles $\theta_1, \theta_2, \theta_3, \theta_4$ and edge lengths l_1, l_2, l_3, l_4 of a quad. We have

$$R^s = \sqrt[3]{\frac{\min\{\theta_1,\theta_2,\theta_3,\theta_4\}}{90^\circ}\left(2 - \frac{\max\{\theta_1,\theta_2,\theta_3,\theta_4\}}{90^\circ}\right)\frac{\min\{l_1,l_2,l_3,l_4\}}{\max\{l_1,l_2,l_3,l_4\}}}. \tag{9}$$

R^s is bounded between 0 and 1. The higher the R^s, the more similar a quad is to a square, regardless of the element size.

Minimizing the number of EPs and cEPs is also important for high-order finite element analysis such as IGA [28]. We assign reward "1" to all regular vertices and penalty "0" to all EPs and cEPs as shown in Fig. 1(c), and then compute the number of EPs (N_{ep}) and close EP pairs (N_{cep}) in a mesh with a total of N_{tot} vertices. R^{ep} is defined as

$$R^{ep} = \left(1 - \frac{N_{ep}}{N_{tot}}\right)\left(1 - \frac{N_{cep}}{N_{tot}}\right). \tag{10}$$

Higher R^{ep} represents sparser EPs, which is preferred in mesh generation. With both R^s and R^{ep} defined, the total reward is evaluated as the sum of the mean and the minimum reward value across all generated quads:

$$R^{fin} = \frac{1}{M}\sum_{i=1}^{M} R_i^s R_i^{ep} + \min\{R_1^s R_1^{ep}, R_2^s R_2^{ep}, \ldots, R_M^s R_M^{ep}\}, \tag{11}$$

where M is the number of quads produced.

The most significant advantage of RL in the SRL-assisted AFM is that we can arbitrarily define the reward function. In this research, we use Eq. (11) to optimize the number and distribution of EPs and quad squareness simultaneously. As training processes, the mesh quality improves by maximizing the reward function, and finally the quality will significantly exceeds previous performance.

4. Numerical results and discussion

The meshes produced by ANSYS serve as the basis for extracting the SL training dataset. In the SL phase, we train four SL policy neural networks $\{\pi_a, \pi_b, \pi_c, \pi_d\}$ using approximately $3.5M$ training rows. Fig. 7 shows SL policy neural network outputs of four held-out examples extracted from the training dataset. With 10 residual blocks, the SL neural networks achieve accuracies of $99.78\%, 98.07\%$ in choosing the reference P_0 and front updating types, and MSE of $0.0017, 0.0037$ in predicting Type 1 and Type 4 new vertices, respectively (Table 1). In these testing cases, π_a and π_b give the correct classification with probabilities of 91.0% and 98.0%. π_c and π_d also give good predictions of new vertices; see the blue and orange points in Fig. 7(c,d). In the RL phase, to generate enough high quality datasets for RL training, we apply the RL Algorithm 2 to five randomly selected complex domains from the 360 input training boundaries in Fig. 1(a). The training starts from SL neural networks $\{\pi_a, \pi_b, \pi_c, \pi_d\}$ and each domain is meshed with many episodes in 24 h to collect high-quality meshes. Over the course of RL training, more than 10,000 meshes are generated for each domain, in which 9% (1130) meshes have their quality exceeding the corresponding SL mesh quality. We keep 500 meshes with the highest mesh quality R^{fin}, and extract dataset from these 500 meshes to train the RL policy neural networks $\{\pi'_a, \pi'_b, \pi'_c, \pi'_d\}$. In both the SL and RL phases, we use stochastic gradient descent as the gradient optimizer [43], and the learning rate is fixed at 10^{-5}.

We apply our SRL-assisted AFM framework to five complicated domains. Each boundary domain inherits some unique challenges. The first curve is shown in Fig. 1(a) with SL and RL meshes. Fig. 8 shows the CMU logo with sharp angles and narrow regions. Due to the fact that the internal logo is composed of multiple components, we specify two fixed sizes for the inner and outer boundaries respectively and arrange seeds for each component instead of using Eq. (1) to avoid zero curvature at sampling points. Our framework meshes the domain successfully within 1 s. In Fig. 9, we take one slice of the segmented mask from a knee MRI dataset [44] and construct four knee joint components (femur, femoral cartilage, tibial cartilage, and tibia) from the mask. The constructed mesh conforms to each other exactly on shared boundaries while exhibiting good adaptivity. The mesh generation for this example takes around 10 s, slightly slower than meshing a whole boundary due to the time required to assign conformal seeds. In Fig. 10, we apply our framework to an airfoil consisting of three sharp-angle components. It takes 8 s to generate an adaptive mesh that effectively preserves sharp angles and narrow regions. Our framework exhibits remarkable efficacy in handling inputs at a large scale while simultaneously reducing the number of elements critical for simulations. In addition, our model supports adding arbitrary numbers of boundary layers. Here, we add two boundary layers to the airfoil boundaries to illustrate the effectiveness of this feature. The last and most complex mesh is the Lake Superior map in Fig. 11. To the best of our knowledge, no ML-based mesh generation method has been attempted on such complex boundaries with multiple holes, sharp angles, narrow regions, and unbalanced seeds before. Our framework managed to mesh the domain in 118 s, showing superior stability in large-scale meshes. All the final meshes preserve correct topology structures, and no intersection detection or post-processing optimization are used when generating these meshes.

Table 2 shows statistics of our resulting meshes. In SRL-assisted AFM, the mesh size is proportional to the seed density. Our mesh size has the same scale as the previous guarantee-quality methods [17–19]. Among the five testing meshes, the proportions of the adopted four updating types f_1, f_2, f_3, f_4 are roughly equivalent to $90\%, 4.5\%, 4.5\%, 1\%$, which means $f_1 \gg f_2 \approx f_3 \gg f_4$. In addition, the numbers of EPs and close EPs are also very stable, around 11% and 7%. In SL meshes, the percentages are around 17% and 12%. Instead of guaranteeing the minimum and maximum angles, the AFM directly uses neural network planning, resulting in slightly poor mesh quality in terms of angle range and Jacobian metric on complex domains with sharp features.

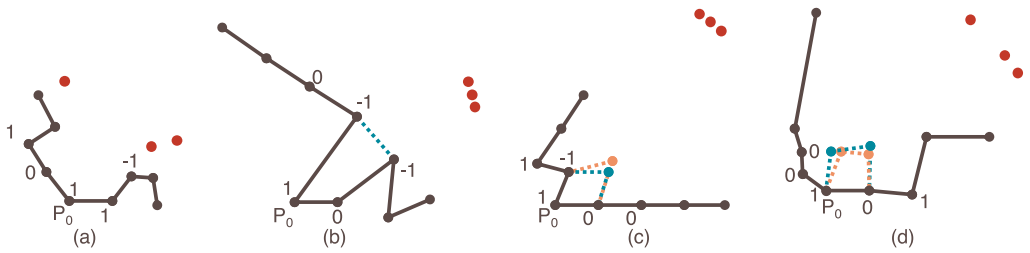


Fig. 7. SL policy neural network outputs of four held-out examples in Table 1. The local vertices $P_0, P_i^r, P_i^l, i = 1, 2, 3, 4$, are in black. Three close vertices $P_i^c, i = 1, 2, 3$, are in red. EP status are listed next to the corresponding vertices. The new vertices predicted by the neural network (if any) are in blue. The new vertices from ground truth (if any) are in orange. (a) π_a accepts P_0 as the reference vertex. (b) π_b selects updating type 2 and two existing points are connected to seal the edge. (c) π_b selects updating type 1, a new point P_1^{new} is generated based on π_c prediction. (d) π_b selects updating type 4, two new points P_1^{new}, P_2^{new} are generated based on π_d predictions.

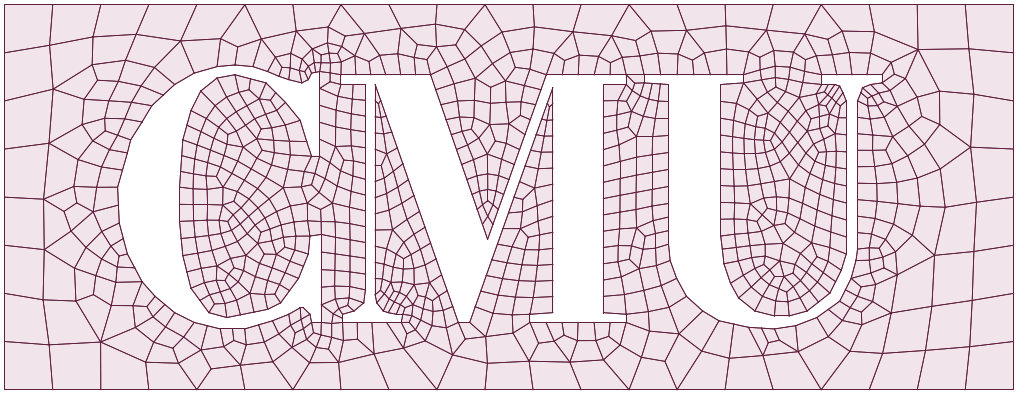


Fig. 8. The CMU logo mesh covering the exterior domain is formed by stripping off the logo in a rectangular box.

Table 1
Examples and statistics of SL neural network outputs with $N = 10$ residual blocks.

Neural network Raw output	$\pi_a(\%)$ $(\mathscr{P}_{acc}, \mathscr{P}_{rej})$	$\pi_b(\%)$ $(\mathscr{P}_1, \mathscr{P}_2, \mathscr{P}_3, \mathscr{P}_4)$	π_c (θ_1, ρ_1)	π_d $(\theta_1, \rho_1), (\theta_2, \rho_2)$
Prediction	(91.0, 9.0)	(1.9, 98.0, 0.0, 0.1)	(0.46, 0.25)	(0.57, 0.18), (0.32, 0.27)
Ground truth	(100.0, 0.0)	(0.0, 100.0, 0.0, 0.0)	(0.55, 0.30)	(0.48, 0.19), (0.28, 0.24)
Accuracy/MSE	99.78	98.07	0.0017	0.0037

Table 2
Statistics of the resulting meshes.

Domain	Mesh size [Vert#, Elem#]	Aspect ratio [Best, Worst]	Valence [EP, cEP[a]]	Angle [Min, Max]	Jacobian [Worst, Best]	Time (s)
Curve	[1361, 1420]	[1.0, 3.8]	[128, 88]	[35°, 148°]	[0.75, 1.0]	0.8
CMU Logo	[924, 1115]	[1.0, 3.5]	[111, 92]	[24°, 140°]	[0.68, 1.0]	0.4
Knee Joint	[2694, 2931]	[1.0, 2.4]	[210, 153]	[27°, 147°]	[0.72, 1.0]	8.1
Air Foil	[2782, 2953]	[1.0, 4.8][b]	[250, 167]	[29°, 150°]	[0.68, 1.0]	8.1
Lake Superior	[12, 150, 11, 618]	[1.0, 7.2]	[389, 223]	[15°, 156°]	[0.60, 1.0]	118.1

[a]cEP is the number of pairs of EPs that are adjacent to each other.
[b]After adding boundary layers, the worst aspect ratio becomes 16.0.

Additionally, the mesh has superior aspect ratio performance compared to guarantee-quality methods due to adaptive seeding in pre-processing period and new vertex prediction by π_c', π_d'. Finally, the meshing time is proportional to the geometry complexity. The Lake Superior example in Fig. 11 is the most complex example because it has massive amount of unbalanced seeds, narrow regions, and sharp angles. Time records reveal that in Algorithm 1, determining the reference vertex (Steps 12–15) is the most time-consuming step since it iterates all vertices on the evolving front to find the proper reference vertex. All the results were computed on a PC with an Intel i7−12700 CPU and 64 GB memory. The code is written in Python and available at https://github.com/CMU-CBML/SRL-AssistedAFM.

5. Conclusion and future work

This paper presents a novel method that automatically generate quad meshes for complex planar domains with four neural networks. The computational framework SRL-assisted AFM integrates the SL-RL algorithm with the AFM.

- We generate a large number of planar domain meshes ($3.5M$ training rows) to increase the diversity of the training dataset. As a result, our framework can mesh new boundaries that are far more complex than those tested in existing ML-based mesh generation methods.

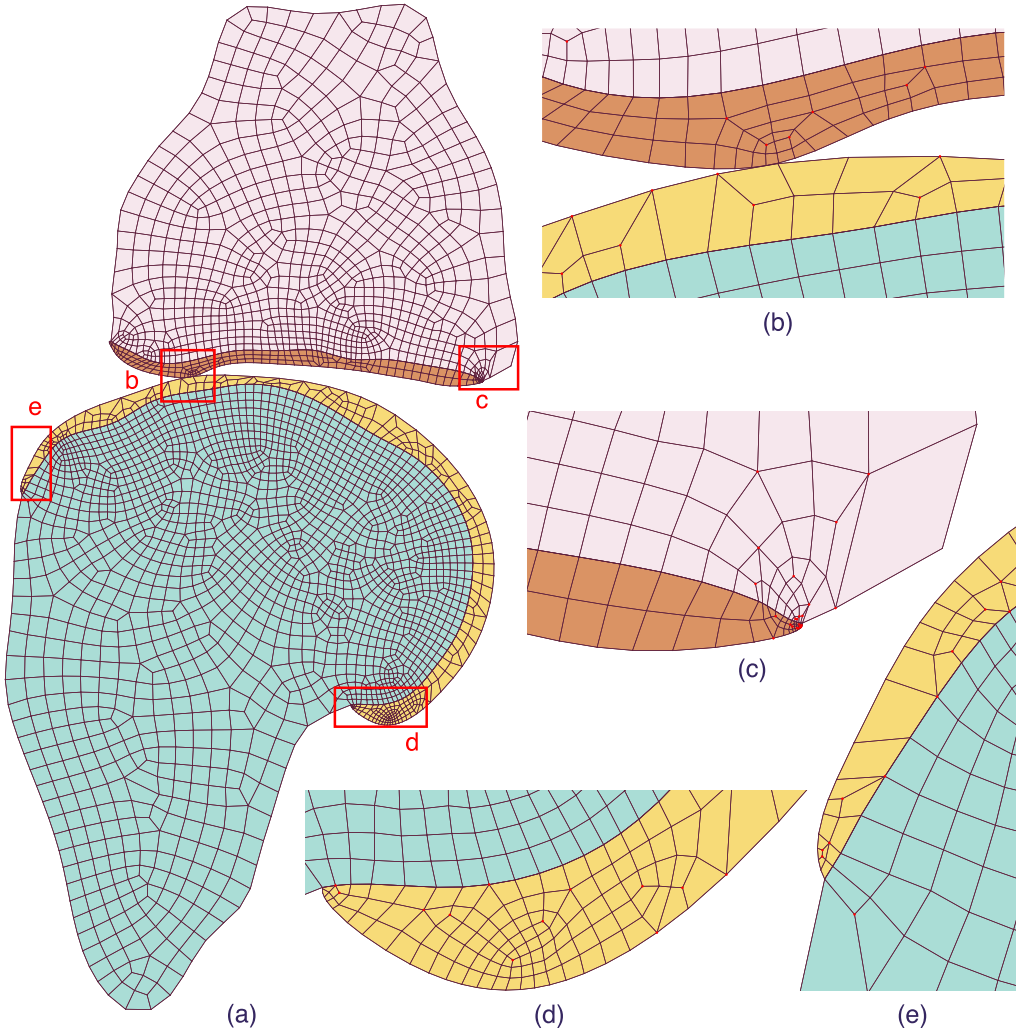


Fig. 9. The knee joint cross-section. (a) Multiple meshes representing the femur (pink), femoral cartilage (orange), tibial cartilage (yellow), and tibia (cyan). (b-e) Zoom-in pictures of red boxes in (a).

- We do not adopt any quality improvement or intersection detection module throughout the pipeline, indicating that our framework can correct errors. This capability significantly simplifies the AFM pipeline and improves the meshing efficiency. For the tested five domains, SRL-assisted AFM generates ~1000 quads per second in average.
- Our SRL-assisted AFM achieves high mesh quality that rivals other methods without the need of post-processing operations or prior knowledge. The framework can generate adaptive meshes to reduce computation burden. The angle range and scaled Jacobian metrics are close to previous guarantee-quality methods. It also significantly improves the aspect ratio and EP penalty metric.
- We define the reward function to reduce the number of EPs and adjacent EPs. The mesh adaptivity is achieved by a size function that assigns seeds according to local front boundary curvature. Users can also assign boundary layers after the mesh is generated.

One limitation of our method is that in the current implementation, the code is written in Python. As a result, it is relatively slower than C/C++. In the future, we will include all modules in C++ to improve computational efficiency. We also aim to extend this SRL-assisted AFM framework to curved surfaces. This is a very promising direction, and we believe the inference performance of the model will improve, supporting more complex input domains. We anticipate that this technology will significantly improve and boost data-driven mesh generation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Code and data availability

The code and datasets generated and analyzed in this paper are accessible in the "SRL-AssistedAFM" GitHub repository. https://github.com/CMU-CBML/SRL-AssistedAFM (DOI: 10.5281/zenodo.8164390). Correspondence for code and data should be addressed to H.T. or Y.J.Z.

Acknowledgments

H. Tong, K. Qian, and Y. J. Zhang were supported in part by the NSF (National Science Foundation), USA grant CMMI-1953323 and a Honda, Japan grant. K. Qian was also supported by Bradford and Diane Smith Graduate Fellowship, USA. This work used RM-node and GPU-node on Bridges-2 Supercomputer at Pittsburgh Supercomputer Center [45,46] through allocation ID eng170006p from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by NSF grants #2138259, #2138286, #2138307, #2137603, and #2138296.

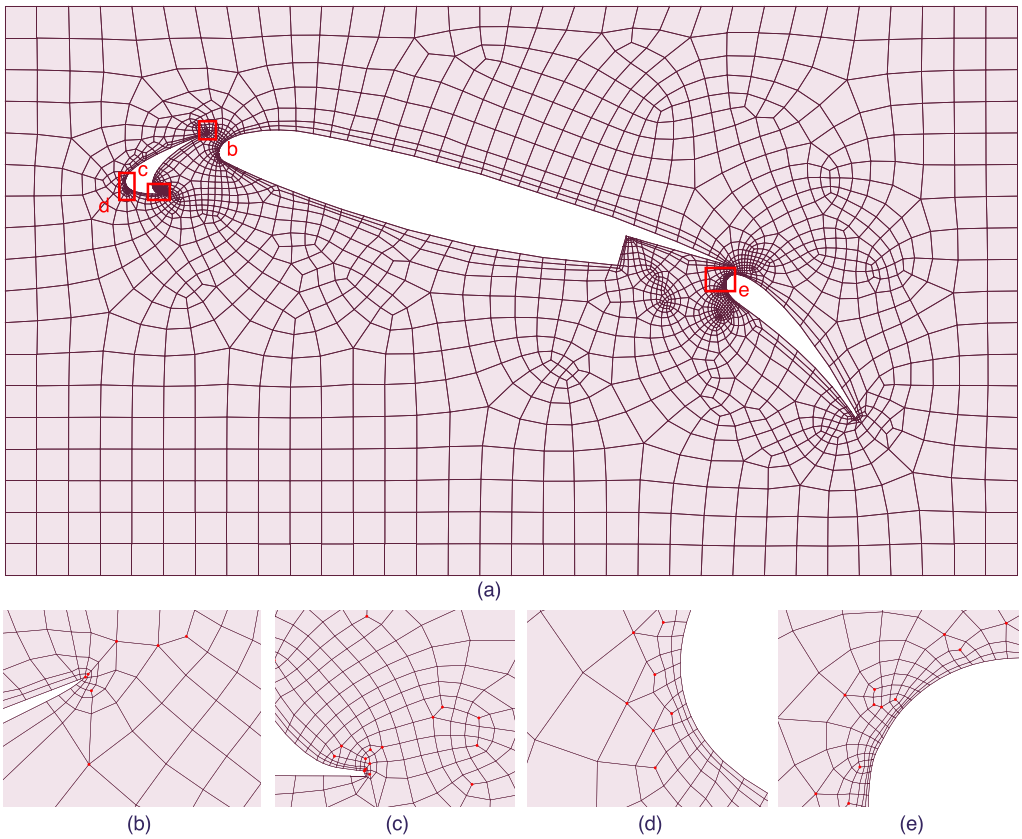


Fig. 10. The airfoil with three components. (a) The mesh covering the exterior domain is formed by stripping off the airfoil in a rectangular box. Two boundary layers are added to the airfoil surfaces. (b-e) Zoom-in pictures of (a).

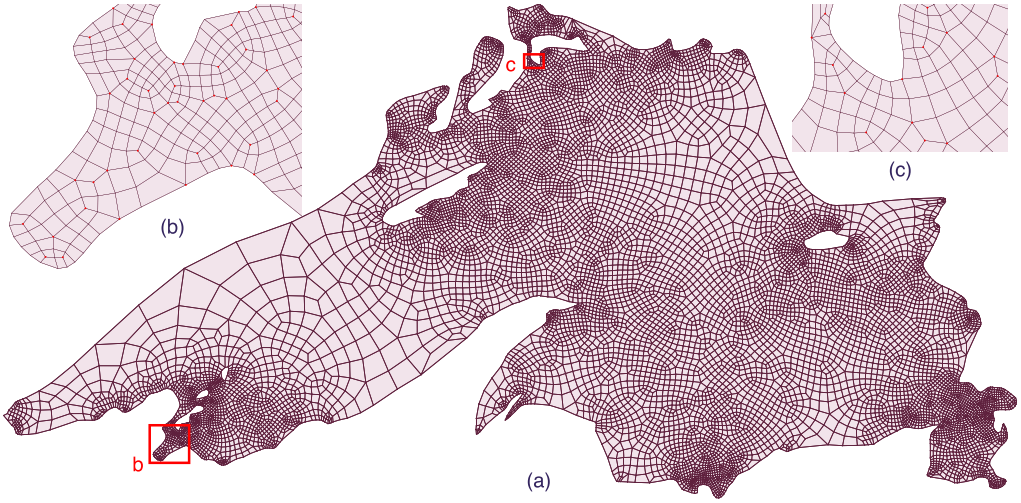


Fig. 11. The Lake Superior map with multiple holes and unbalanced seeds. (a) Final all-quad mesh. (b-c) Zoom-in pictures of the red boxes in (a).

References

[1] J. Slotnick, A. Khodadoust, J. Alonso, D. Darmofal, W. Gropp, E. Lurie, D. Mavriplis, CFD Vision 2030 Study: a Path to Revolutionary Computational Aerosciences, NASA, Washington, DC, USA, 2013.

[2] Y. Zhang, Geometric Modeling and Mesh Generation from Scanned Images, CRC Press, Taylor & Francis Group, 2016.

[3] Y. Zhang, Challenges and advances in image-based geometric modeling and mesh generation, Image-Based Geom. Model. Mesh Gener. (2013).

[4] J.-F. Remacle, F. Henrotte, T. Carrier-Baudouin, E. Béchet, E. Marchandise, C. Geuzaine, T. Mouton, A frontal Delaunay quad mesh generator using the L_∞ norm, Int. J. Numer. Methods Eng. 94 (5) (2013) 494–512.

[5] C. Liu, W. Yu, Z. Chen, X. Li, Distributed poly-square mapping for large-scale semi-structured quad mesh generation, Comput. Aided Des. 90 (2017) 5–17.

[6] C.S. Verma, K. Suresh, A robust combinatorial approach to reduce singularities in quadrilateral meshes, Procedia Eng. 124 (2015) 252–264.

[7] J. Docampo-Sanchez, R. Haimes, Towards fully regular quad mesh generation, in: AIAA Scitech Forum, 2019, p. 1988.

[8] C.S. Verma, K. Suresh, αMST: a robust unified algorithm for quadrilateral mesh adaptation, Procedia Eng. 163 (2016) 238–250.

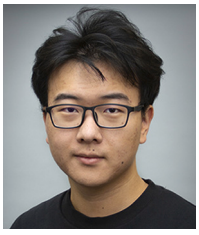
[9] J.F. Thompson, B.K. Soni, N.P. Weatherill, Handbook of Grid Generation, CRC Press, 1998.

[10] S.J. Owen, A survey of unstructured mesh generation technology, Int. Meshing Roundtable 239 (1998) 267.

[11] S.-H. Teng, C.W. Wong, Unstructured mesh generation: theory, practice, and perspectives, Internat. J. Comput. Geom. Appl. 10 (03) (2000) 227–266.

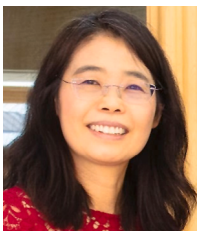
[12] C. Lee, S. Lo, A new scheme for the generation of a graded quadrilateral mesh, Comput. Struct. 52 (5) (1994) 847–857.
[13] S.J. Owen, M.L. Staten, S.A. Canann, S. Saigal, Q-Morph: an indirect approach to advancing front quad meshing, Internat. J. Numer. Methods Engrg. 44 (9) (1999) 1317–1340.
[14] T.D. Blacker, M.B. Stephenson, Paving: a new approach to automated quadrilateral mesh generation, Internat. J. Numer. Methods Engrg. 32 (4) (1991) 811–847.
[15] C.-H. Peng, M. Barton, C. Jiang, P. Wonka, Exploring quadrangulations, ACM Trans. Graph. 33 (1) (2014) 1–13.
[16] P.L. Baehmann, S.L. Wittchen, M.S. Shephard, K.R. Grice, M.A. Yerry, Robust, geometrically based, automatic two-dimensional mesh generation, Internat. J. Numer. Methods Engrg. 24 (6) (1987) 1043–1078.
[17] X. Liang, M.S. Ebeida, Y. Zhang, Guaranteed-quality all-quadrilateral mesh generation with feature preservation, Comput. Methods Appl. Mech. Engrg. 199 (29–32) (2010) 2072–2083.
[18] X. Liang, Y. Zhang, Hexagon-based all-quadrilateral mesh generation with guaranteed angle bounds, Comput. Methods Appl. Mech. Engrg. 200 (23–24) (2011) 2005–2020.
[19] X. Liang, Y. Zhang, Matching interior and exterior all-quadrilateral meshes with guaranteed angle bounds, Eng. Comput. 28 (4) (2012) 375–389.
[20] Y. Zhang, C. Bajaj, G. Xu, Surface smoothing and quality improvement of quadrilateral/hexahedral meshes with geometric flow, Commun. Numer. Methods. Eng. 25 (1) (2009) 1–18.
[21] Y. Zhang, C. Bajaj, B.-S. Sohn, 3D finite element meshing from imaging data, Comput. Methods Appl. Mech. Engrg. 194 (48–49) (2005) 5083–5106.
[22] Y. Zhang, C. Bajaj, Adaptive and quality quadrilateral/hexahedral meshing from volumetric data, Comput. Methods Appl. Mech. Engrg. 195 (9–12) (2006) 942–960.
[23] Y. Zhang, T.J. Hughes, C.L. Bajaj, An automatic 3D mesh generation method for domains with multiple materials, Comput. Methods Appl. Mech. Engrg. 199 (5–8) (2010) 405–415.
[24] S. Dong, S. Kircher, M. Garland, Harmonic functions for quadrilateral remeshing of arbitrary manifolds, Comput. Aided Geom. Design 22 (5) (2005) 392–423.
[25] Y. Tong, P. Alliez, D. Cohen-Steiner, M. Desbrun, Designing quadrangulations with discrete harmonic forms, in: Eurographics Symposium on Geometry Processing, 2006.
[26] D. Bommes, H. Zimmer, L. Kobbelt, Mixed-integer quadrangulation, ACM Trans. Graph. 28 (3) (2009) 1–10.
[27] D. Bommes, M. Campen, H.-C. Ebke, P. Alliez, L. Kobbelt, Integer-grid maps for reliable quad meshing, ACM Trans. Graph. 32 (4) (2013) 1–12.
[28] X. Wei, X. Li, K. Qian, T.J. Hughes, Y.J. Zhang, H. Casquero, Analysis-suitable unstructured T-splines: multiple extraordinary points per face, Comput. Methods Appl. Mech. Engrg. 391 (2022) 114494.
[29] X. Wei, Y. Zhang, T.J. Hughes, M.A. Scott, Truncated hierarchical Catmull-Clark subdivision with local refinement, Comput. Methods Appl. Mech. Engrg. 291 (2015) 1–20.
[30] X. Wei, X. Li, Y.J. Zhang, T.J. Hughes, Tuned hybrid nonuniform subdivision surfaces with optimal convergence rates, Internat. J. Numer. Methods Engrg. 122 (9) (2021) 2117–2144.
[31] X. Wei, Y. Zhang, L. Liu, T.J. Hughes, Truncated T-splines: fundamentals and methods, Comput. Methods Appl. Mech. Engrg. 316 (2017) 349–372.
[32] W. Wang, Y. Zhang, M.A. Scott, T.J. Hughes, Converting an unstructured quadrilateral mesh to a standard T-spline surface, Comput. Mech. 48 (2011) 477–498.
[33] W. Wang, Y. Zhang, G. Xu, T.J. Hughes, Converting an unstructured quadrilateral/hexahedral mesh to a rational T-spline, Comput. Mech. 50 (2012) 65–84.
[34] R. Löhner, P. Parikh, Generation of three-dimensional unstructured grids by the advancing-front method, Internat. J. Numer. Methods Fluids 8 (10) (1988) 1135–1149.
[35] Y. Guo, X. Huang, Z. Ma, Y. Hai, R. Zhao, K. Sun, An improved advancing-front-Delaunay method for triangular mesh generation, in: 38th Computer Graphics International Conference, 2021, pp. 477–487.
[36] J. Pan, J. Huang, G. Cheng, Y. Zeng, Reinforcement learning for automatic quadrilateral mesh generation: A soft actor–critic approach, Neural Netw. 157 (2023) 288–304.
[37] P. Lu, N. Wang, Y. Lin, X. Zhang, Y. Wu, H. Zhang, A new unstructured hybrid mesh generation method based on BP-ANN, in: Journal of Physics: Conference Series, Vol. 2280, 2022, 012045.
[38] E. Seveno, et al., Towards an adaptive advancing front method, in: 6th International Meshing Roundtable, 1997, pp. 349–362.
[39] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, Neural Netw. 2 (5) (1989) 359–366.
[40] A.R. Cassandra, A survey of POMDP applications, in: Working Notes of AAAI 1998 Fall Symposium on Planning with Partially Observable Markov Decision Processes, Vol. 1724, 1998.
[41] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
[42] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 2980–2988.
[43] L. Bottou, Stochastic gradient descent tricks, Neural Netw.: Tricks Trade (2012) 421–436.
[44] F. Ambellan, A. Tack, M. Ehlke, S. Zachow, Automated segmentation of knee bone and cartilage combining statistical shape knowledge and convolutional neural networks: data from the osteoarthritis initiative, Med. Image Anal. 52 (2019) 109–118.
[45] J. Towns, T. Cockerill, M. Dahan, I. Foster, K. Gaither, A. Grimshaw, V. Hazlewood, S. Lathrop, D. Lifka, G.D. Peterson, R. Roskies, J.R. Scott, N. Wilkins-Diehr, XSEDE: accelerating scientific discovery, Comput. Sci. Eng. 16 (5) (2014) 62–74.
[46] N. Wilkins-Diehr, S. Sanielevici, J. Alameda, J. Cazes, L. Crosby, M. Pierce, R. Roskies, An overview of the XSEDE extended collaborative support program, in: High Performance Computer Applications - 6th International Conference, ISUM 2015, in: Communications in Computer and Information Science, vol. 595, 2016, pp. 3–13.

Hua Tong received his B.S. in Engineering Science from the University of Science and Technology of China in 2022. He is currently a Ph.D. student in the Department of Mechanical Engineering at Carnegie Mellon University. His research interests include mesh generation and machine learning.



Kuanren Qian holds a B.S. in Mechanical Engineering from the University of California, San Diego in 2018 and a M.S. in Mechanical Engineering – Research from Carnegie Mellon University in 2020. He is a Ph.D. student in the Department of Mechanical Engineering at Carnegie Mellon University. His research interests include isogeometric analysis, neuron growth simulation, phase field method, machine learning, and physics-informed modeling.

Eni Halilaj received her B.A. in Engineering and Italian Studies in 2008 and Ph.D. in Biomedical Engineering in 2015 from Brown University. She became an assistant professor in Mechanical Engineering at Carnegie Mellon University with courtesy appointments in Biomedical Engineering and Robotics Institute in 2018. Her research interests include biomechanics, imaging, modeling, orthopedics, and osteoarthritis.



Yongjie Jessica Zhang holds B.Eng. in Automotive Engineering and M.Eng. in Engineering Mechanics from Tsinghua University; and M.Eng. in Aerospace Engineering and Engineering Mechanics and Ph.D. in Computational Engineering and Sciences from the Institute for Computational Engineering and Sciences, The University of Texas at Austin. She is the George Tallman Ladd and Florence Barrett Ladd Professor of Mechanical Engineering, Carnegie Mellon University with a courtesy appointment in Biomedical Engineering. Her research interests include computational geometry, isogeometric analysis, finite element method, data-driven modeling, image processing and mesh generation.